

DSA Mock 2 FEB 2026

vaibhavkharche4328@gmail.com [Switch accounts](#)



* Indicates required question

Email *

☐

Record vaibhavkharche4328@gmail.com as the email to be included with my response

Name *

Your answer

PRN *

Your answer



Q.1 You are tasked with solving a complex business problem. Before you begin, which of the following steps is most critical for ensuring that you address the core issue rather than just symptoms? ★ 1 point

Snippet: A manager approaches you with a performance decline in a team. The first step in solving this problem is critical for effective resolution.

- ☐ A) Generate potential solutions immediately.
- ☐ B) Identify the specific problem and understand its root cause.
- ☐ C) Evaluate potential consequences of each solution.
- ☐ D) Identify who is responsible for the problem

Q.2 Which of the following best describes the concept of "problem definition" in problem solving? ★ 1 point

Snippet: Problem definition is often misunderstood, leading to solving issues that are not the real problem at hand.

- ☐ A) The process of generating multiple solutions to a given challenge.
- ☐ B) The phase where the goal is to outline the problem clearly and ensure it is understood.
- ☐ C) A process where potential stakeholders are involved to resolve the problem.
- ☒ D) A process of identifying the resources required for the solution.



Q.3 When defining a problem in a business context, why is it important to distinguish ★ 1 point

between the symptom of the problem and the underlying cause?

Snippet: In problem solving, addressing the root cause of an issue often leads to more sustainable solutions.

- ☐ A) It helps avoid focusing on irrelevant details.
- ☐ B) It leads to a quicker resolution by ignoring the underlying factors.
- ☐ C) It ensures that all issues related to the symptom are solved, even if not the cause.
- ☐ D) It ensures the problem will not reoccur by addressing both symptom and cause.

Q.4 What is a fundamental principle of problem-solving that helps guide a team from ★ 1 point

defining a problem to implementing a solution?

Snippet: Problem-solving strategies often follow a cycle, starting with understanding the problem and ending with a practical solution.

- ☐ A) Defining the problem and immediately implementing solutions based on past experiences.
- ☐ B) Analyzing and framing the problem, generating alternative solutions, and then evaluating them critically.
- ☐ C) Starting with a predefined solution and adjusting the problem definition accordingly.
- ☐ D) Focusing on evaluating potential solutions first before identifying the problem itself.
- ☐ Option 5



Q. 5 During the problem-solving process, which of the following best describes the "problem identification" stage?

* 1 point

Snippet: The problem identification phase ensures that the real issue is recognized, providing a clear direction for solution development.

- ☐ A) Generating and testing various potential solutions to the problem.
- ☐ B) Understanding the issue, gathering relevant data, and identifying its true nature and scope.
- ☐ C) Implementing the first solution that seems viable.
- ☐ D) Evaluating the outcome of the chosen solution and making necessary adjustments.

Q. 6 Which of the following best describes the primary disadvantage of using an array to implement a queue?

* 1 point

Arrays are simple to implement, but using them for queues may lead to inefficiencies, especially when performing enqueue and dequeue operations frequently, as resizing and shifting elements can be costly.

- ☐ A) Arrays provide inefficient memory usage when elements are inserted or removed.
- ☐ B) Arrays do not allow for random access, making it difficult to access elements in a queue.
- ☐ C) Arrays require dynamic resizing when the queue grows, causing performance degradation.
- ☐ D) Arrays require shifting elements when performing dequeue operations, leading to $O(n)$ complexity for each operation.



Q. 7 What is the Big O time complexity of merging two sorted arrays, each of size n , into a single sorted array? ★ 1 point

Merging two sorted arrays is a common operation in divide-and-conquer algorithms like merge sort. The time complexity depends on the number of elements involved and how they are combined.

- ☐ A) $O(n)$
- ☐ B) $O(n \log n)$
- ☐ C) $O(n^2)$
- ☐ D) $O(2n)$

Q.8 In the context of Abstract Data Types (ADTs), which of the following statements is true about the design of a stack? ★ 1 point

A stack is a Last In, First Out (LIFO) data structure. It is an ADT because its design specifies what operations are supported (push, pop, peek) and not how these operations are implemented.

- ☐ A) A stack must always be implemented using arrays, as they are the only data structure that supports LIFO operations efficiently.
- ☐ B) A stack is an ADT that supports random access and allows modification of elements in any order.
- ☐ C) A stack defines operations for adding and removing elements in a LIFO manner, but its internal implementation can vary.
- ☐ D) A stack is a queue where elements are removed in a first-come, first-served order.



Q. 9 Which of the following is a key feature of a circular queue that differentiates it from a regular queue?

* 1 point

A regular queue can waste memory if elements are dequeued from the front and the rear remains empty, while a circular queue allows the rear pointer to wrap around to the front, making efficient use of available space.

- ☐ A) A circular queue allows random access to any element.
- ☐ B) A circular queue ensures that all enqueue and dequeue operations are $O(1)$ regardless of the size of the queue.
- ☐ C) A circular queue uses a fixed-size array and wraps around when the rear reaches the end of the array.
- ☐ D) A circular queue requires dynamic resizing as the number of elements increases.

Q. 10 What is the primary advantage of using Big O notation in algorithm analysis?

* 1 point

Big O notation allows us to abstract away specific machine-level details and focuses on the performance of an algorithm in terms of input size, helping developers compare algorithms in a language-agnostic manner.

- ☐ A) It helps developers understand how long an algorithm will run on their specific hardware.
- ☐ B) It provides a way to measure the constant factors of an algorithm's performance.
- ☐ C) It allows developers to estimate the worst-case, average-case, and best-case running time of an algorithm independently of hardware.es
- ☐ D) It enables the estimation of space complexity with respect to time complexity.



Q. 11: Which of the following is a significant advantage of a doubly linked list over a singly linked list? ★ 1 point

A doubly linked list allows traversal in both directions (forward and backward) by maintaining pointers to both the next and previous nodes, which can make certain operations more efficient.

- ☐ A) A doubly linked list requires less memory because it only stores data.
- ☐ B) A doubly linked list allows traversal in both directions, making deletion of a node more efficient when we have a reference to it.
- ☐ C) A doubly linked list is always faster for insertion and deletion than a singly linked list.
- ☐ D) A doubly linked list does not require additional pointers, making it more space-efficient than a singly linked list

Q. 11: ★ 1 point

Which of the following is a significant advantage of a doubly linked list over a singly linked list?

A doubly linked list allows traversal in both directions (forward and backward) by maintaining pointers to both the next and previous nodes, which can make certain operations more efficient.

- ☐ A) A doubly linked list requires less memory because it only stores data.
- ☐ B) A doubly linked list allows traversal in both directions, making deletion of a node more efficient when we have a reference to it.
- ☐ C) A doubly linked list is always faster for insertion and deletion than a singly linked list.
- ☐ D) A doubly linked list does not require additional pointers, making it more space-efficient than a singly linked list.



Q. 12:*** 1 point**

In which of the following situations is a circular linked list most advantageous compared to a regular (non-circular) linked list?

Circular linked lists have a special structure where the last node points back to the first node, which can be advantageous in certain applications, particularly in situations where traversal from any node to the next is required continuously.

- ☐ A) When elements need to be processed in a linear fashion and no element needs to loop back.
- ☐ B) When the list needs to be traversed starting from any point and returning to the starting point after reaching the last node.
- ☐ C) When we need fast random access to elements by index.
- ☐ D) When memory usage should be minimized by avoiding node pointers.



Q. 13:*** 1 point**

Which of the following is the primary disadvantage of using a linked list over an array for storing a collection of elements?

Linked lists have different performance characteristics than arrays, especially when considering memory usage and the need to access elements by index.

- ☐ A) Linked lists provide constant-time access to any element, unlike arrays, which are slower for direct access.
- ☐ B) Linked lists require more memory for storing the data due to the need for additional pointers for each element.
- ☐ C) Arrays are more dynamic and can adjust their size more efficiently than linked lists.
- ☐ D) Linked lists provide faster insertion and deletion at arbitrary positions compared to arrays.



Q. 14:

* 1 point

Which of the following best describes a key advantage of using a linked list over an array in terms of memory management?

A linked list is a dynamic data structure where elements can be inserted or deleted without requiring a contiguous block of memory, which can be advantageous in memory-constrained environments.

- ☐ A) Linked lists always occupy less memory than arrays because they don't need to store extra information.
- ☐ B) Linked lists use contiguous memory for storing elements, making them faster to allocate and deallocate.
- ☐ C) Linked lists allow efficient dynamic memory allocation and do not require predefined memory sizes, unlike arrays.
- ☐ D) Linked lists can be resized automatically to fit the number of elements, unlike arrays.

Which of the following is a significant advantage of a doubly linked list over a singly linked list?

* 1 point

A doubly linked list allows traversal in both directions (forward and backward) by maintaining pointers to both the next and previous nodes, which can make certain operations more efficient.

- ☐ A) A doubly linked list requires less memory because it only stores data.
- ☐ B) A doubly linked list allows traversal in both directions, making deletion of a node more efficient when we have a reference to it.
- ☐ C) A doubly linked list is always faster for insertion and deletion than a singly linked list.
- ☐ D) A doubly linked list does not require additional pointers, making it more space-efficient than a singly linked list.



In which of the following situations is a circular linked list most advantageous compared to a regular (non-circular) linked list?

* 1 point

Circular linked lists have a special structure where the last node points back to the first node, which can be advantageous in certain applications, particularly in situations where traversal from any node to the next is required continuously.

- ☐ A) When elements need to be processed in a linear fashion and no element needs to loop back.
- ☐ B) When the list needs to be traversed starting from any point and returning to the starting point after reaching the last node.
- ☐ C) When we need fast random access to elements by index.
- ☐ D) When memory usage should be minimized by avoiding node pointers.

Which of the following is the primary disadvantage of using a linked list over an array for storing a collection of elements?

* 1 point

Linked lists have different performance characteristics than arrays, especially when considering memory usage and the need to access elements by index.

- ☐ A) Linked lists provide constant-time access to any element, unlike arrays, which are slower for direct access.
- ☐ B) Linked lists require more memory for storing the data due to the need for additional pointers for each element.
- ☐ C) Arrays are more dynamic and can adjust their size more efficiently than linked lists.
- ☐ D) Linked lists provide faster insertion and deletion at arbitrary positions compared to arrays.



Which of the following best describes a key advantage of using a linked list over an array in terms of memory management? * 1 point

A linked list is a dynamic data structure where elements can be inserted or deleted without requiring a contiguous block of memory, which can be advantageous in memory-constrained environments.

- ☐ A) Linked lists always occupy less memory than arrays because they don't need to store extra information.
- ☐ B) Linked lists use contiguous memory for storing elements, making them faster to allocate and deallocate.
- ☐ C) Linked lists allow efficient dynamic memory allocation and do not require predefined memory sizes, unlike arrays.
- ☐ D) Linked lists can be resized automatically to fit the number of elements, unlike arrays.

When should you prefer using an array over a linked list? * 1 point

Arrays and linked lists each have their strengths and weaknesses. Understanding when to use one over the other is crucial for efficient memory and performance management in programming.

- ☐ A) When you need fast random access to elements by index and the size of the collection is known beforehand.
- ☐ B) When the collection size can grow or shrink dynamically, and elements need to be inserted or deleted frequently
- ☐ C) When you need to traverse the collection in a circular manner, such as in a round-robin scheduler.
- ☐ D) When elements need to be added or removed at the end of the list frequently, and there's no need for fast indexing.



Which of the following is the primary function of a base condition in a recursive function?

* 1 point

In recursion, a base condition serves as the stopping criterion to avoid infinite recursion and to ensure that the recursion halts at the correct point.

- ☐ A) To terminate the function after a predefined number of recursive calls.
- ☐ B) To prevent the function from being called with invalid arguments.
- ☐ C) To halt the recursion and return a value when a certain condition is met, preventing infinite recursion.
- ☐ D) To optimize the recursion by eliminating unnecessary function calls.

Which of the following best describes indirect recursion?

* 1 point

Indirect recursion involves a function calling another function, which, in turn, calls the first function, leading to a cycle. This is different from direct recursion, where a function directly calls itself.

- ☐ A) Direct recursion occurs when a function calls itself with no intermediary function.
- ☐ B) Indirect recursion occurs when multiple functions call each other in a cycle, leading to recursion.
- ☐ C) Indirect recursion can only occur when two functions are involved in the recursive cycle.
- ☐ D) Indirect recursion is more efficient than direct recursion in all scenarios.



Which of the following is a key advantage of using an AVL tree over a regular binary search tree (BST)?

* 1 point

AVL trees are a type of self-balancing binary search tree. They maintain a balance factor to ensure that the tree remains balanced, leading to better performance in terms of search complexity.

- ☐ A) AVL trees do not require additional memory for balancing information.
- ☐ B) AVL trees automatically balance themselves, ensuring that the search time complexity is always $O(\log n)$.
- ☐ C) AVL trees always use more memory compared to a regular BST, making them inefficient.
- ☐ D) AVL trees do not support the same search and insertion operations as regular binary search trees.

Which tree traversal method visits the nodes in the order of Left-Root-Right?

* 1 point

Tree traversal methods define the order in which nodes are visited. Different methods are useful in different scenarios depending on the task at hand.

- ☐ A) Pre-order traversal
- ☐ B) In-order traversal
- ☐ C) Post-order traversal
- ☐ D) Level-order traversal



What is the main limitation of a binary search tree (BST) that can lead to poor performance in certain scenarios? * 1 point

Binary search trees rely on their structure to maintain efficient search operations. However, their structure can become unbalanced, leading to inefficient performance.

- ☐ A) The search time in a BST is always $O(\log n)$, regardless of the shape of the tree.
- ☐ B) A BST can become unbalanced, leading to a worst-case search time of $O(n)$ when the tree degenerates into a linked list.
- ☐ C) A BST always requires additional balancing, which increases its memory usage.
- ☐ D) A BST is inherently slow for search operations due to its hierarchical structure

Which of the following is a key advantage of using an AVL tree over a regular binary search tree (BST)? * 1 point

AVL trees are a type of self-balancing binary search tree. They maintain a balance factor to ensure that the tree remains balanced, leading to better performance in terms of search complexity.

- ☐ A) AVL trees do not require additional memory for balancing information.
- ☐ B) AVL trees automatically balance themselves, ensuring that the search time complexity is always $O(\log n)$.
- ☐ C) AVL trees always use more memory compared to a regular BST, making them inefficient.
- ☐ D) AVL trees do not support the same search and insertion operations as regular binary search trees.



Which tree traversal method visits the nodes in the order of Left-Root-Right?

* 1 point

Tree traversal methods define the order in which nodes are visited. Different methods are useful in different scenarios depending on the task at hand.

- ☐ A) Pre-order traversal
- ☐ B) In-order traversal
- ☐ C) Post-order traversal
- ☐ D) Level-order traversal

What is the main limitation of a binary search tree (BST) that can lead to poor performance in certain scenarios?

* 1 point

Binary search trees rely on their structure to maintain efficient search operations. However, their structure can become unbalanced, leading to inefficient performance.

- ☐ A) The search time in a BST is always $O(\log n)$, regardless of the shape of the tree.
- ☐ B) A BST can become unbalanced, leading to a worst-case search time of $O(n)$ when the tree degenerates into a linked list.
- ☐ C) A BST always requires additional balancing, which increases its memory usage.
- ☐ D) A BST is inherently slow for search operations due to its hierarchical structure.



Which of the following is a valid application of binary trees in computer science?

* 1 point

Binary trees, particularly binary search trees (BST), are widely used in applications that require efficient searching, sorting, and hierarchical data representation.

- ☐ A) Representing hierarchical data such as a file system.
- ☐ B) Storing data for fast access where insertion and deletion are frequent but searches are rare.
- ☐ C) Implementing efficient search algorithms in unsorted data sets.
- ☐ D) Storing unstructured data with no defined relationships

Which of the following is a key advantage of using an AVL tree over a regular binary search tree (BST)?

* 1 point

AVL trees are a type of self-balancing binary search tree. They maintain a balance factor to ensure that the tree remains balanced, leading to better performance in terms of search complexity.

- ☐ A) AVL trees do not require additional memory for balancing information.
- ☐ B) AVL trees automatically balance themselves, ensuring that the search time complexity is always $O(\log n)$.
- ☐ C) AVL trees always use more memory compared to a regular BST, making them inefficient.
- ☐ D) AVL trees do not support the same search and insertion operations as regular binary search trees.



Which tree traversal method visits the nodes in the order of Left-Root-Right?

* 1 point

Tree traversal methods define the order in which nodes are visited. Different methods are useful in different scenarios depending on the task at hand.

- ☐ A) Pre-order traversal
- ☐ B) In-order traversal
- ☐ C) Post-order traversal
- ☐ D) Level-order traversal

What is the main limitation of a binary search tree (BST) that can lead to poor performance in certain scenarios?

* 1 point

Binary search trees rely on their structure to maintain efficient search operations. However, their structure can become unbalanced, leading to inefficient performance.

- ☐ A) The search time in a BST is always $O(\log n)$, regardless of the shape of the tree.
- ☐ B) A BST can become unbalanced, leading to a worst-case search time of $O(n)$ when the tree degenerates into a linked list.
- ☐ C) A BST always requires additional balancing, which increases its memory usage.
- ☐ D) A BST is inherently slow for search operations due to its hierarchical structure.



Which of the following is a valid application of binary trees in computer science?

* 1 point

Binary trees, particularly binary search trees (BST), are widely used in applications that require efficient searching, sorting, and hierarchical data representation.

- ☐ A) Representing hierarchical data such as a file system.
- ☐ B) Storing data for fast access where insertion and deletion are frequent but searches are rare.
- ☐ C) Implementing efficient search algorithms in unsorted data sets.
- ☐ D) Storing unstructured data with no defined relationships.

How can the limitations of binary search trees (BSTs), such as poor performance due to unbalanced structures, be overcome?

* 1 point

There are several techniques to ensure that binary search trees remain balanced, which guarantees optimal search, insert, and delete operations.

- ☐ A) By converting the BST into a hash table, ensuring constant-time complexity for all operations.
- ☐ B) By converting the BST into a binary heap to improve the balance of the tree.
- ☐ C) By using self-balancing trees such as AVL trees or Red-Black trees, which automatically maintain balance during operations.
- ☐ D) By limiting the number of nodes in the BST to ensure that it does not grow too large.



Which of the following best describes the primary advantage of using binary search over sequential search?

* 1 point

Binary search is more efficient than sequential search, but it requires the input data to be sorted, which limits its applicability in certain cases.

- ☐ A) Binary search works well for unsorted data, making it more flexible than sequential search.
- ☐ B) Binary search reduces the time complexity to $O(\log n)$ compared to the $O(n)$ complexity of sequential search, but only if the data is sorted.
- ☐ C) Binary search requires less memory and can perform better in all cases compared to sequential search.
- ☐ D) Binary search can work faster than sequential search even when the data is unsorted.

Which of the following sorting algorithms has the best average-case time complexity?

* 1 point

Sorting algorithms vary in terms of time complexity depending on the input data and the nature of the algorithm's design.

- ☐ A) Bubble sort
- ☐ B) Selection sort
- ☐ C) Quicksort
- ☐ D) Insertion sort



What is the primary reason why quicksort is preferred over other sorting algorithms in practice, despite its worst-case time complexity of $O(n^2)$? * 1 point

Quicksort is widely used due to its average-case performance and its ability to efficiently sort large datasets, even though its worst-case performance is less ideal.

- ☐ A) Quicksort is a stable sorting algorithm, making it ideal for sorting complex data types.
- ☐ B) Quicksort has an average-case time complexity of $O(n \log n)$, which makes it faster in most practical scenarios compared to other $O(n \log n)$ algorithms.
- ☐ C) Quicksort has a time complexity of $O(n)$ in all cases, making it the most efficient algorithm in practice.
- ☐ D) Quicksort is simple to implement and does not require additional memory.

Which sorting algorithm is known for having the worst-case time complexity of $O(n^2)$ and is not suitable for large datasets, but is very efficient for small datasets or nearly sorted data? * 1 point

Certain algorithms have a better practical performance for small datasets, even if their worst-case performance is poor for larger datasets.

- ☐ A) Merge sort
- ☐ B) Bubble sort
- ☐ C) Quick sort
- ☒ D) Insertion sort



When analyzing sorting algorithms, how does the nature of the input data (e.g., nearly sorted or reversed) affect the choice of algorithm? * 1 point

Some algorithms perform much better than others depending on the characteristics of the input data, such as whether it is already sorted, nearly sorted, or completely reversed.

- ☐ A) Algorithms like merge sort and heapsort always perform optimally, regardless of the data order.
- ☐ B) Insertion sort performs very well on nearly sorted data, whereas algorithms like quicksort can degrade in performance when the data is reversed.
- ☐ C) Algorithms such as quicksort and selection sort always perform in $O(n \log n)$ time, regardless of the input order.
- ☐ D) The choice of algorithm is not affected by the data order, as all algorithms have the same time complexity.

In which of the following scenarios would you prefer to use selection sort over more efficient algorithms like quicksort or mergesort? * 1 point

Selection sort is not the most efficient sorting algorithm, but it might be used in certain situations where its simple implementation and certain data characteristics make it viable.

- ☐ A) When the dataset is already sorted.
- ☐ B) When memory space is extremely limited, and the algorithm must run in-place with constant space complexity.
- ☐ C) When sorting large datasets where performance and time complexity are critical.
- ☐ D) When you need a stable sorting algorithm and the dataset has many duplicate elements



Which of the following is the primary advantage of using a hash table for search operations compared to other search techniques such as binary search? * 1 point

Hash tables offer constant time complexity on average for search operations, making them more efficient than other techniques for large datasets under typical conditions.

- ☐ A) Hash tables provide guaranteed $O(\log n)$ time complexity for both search and insert operations, regardless of the data.
- ☐ B) Hash tables reduce the time complexity to $O(1)$ for search operations, assuming good hash function distribution and low collisions.
- ☐ C) Binary search trees are always faster than hash tables for searching, especially for unsorted data.
- ☐ D) Hash tables automatically balance themselves, making them ideal for storing sorted data.

What is the main problem with using simple hash functions, and how does it affect the performance of a hash table? * 1 point

The choice of hash function is critical to the performance of a hash table. A poor hash function can cause many keys to map to the same index, resulting in inefficient search and insertion operations.

- ☐ A) A bad hash function can lead to clustering, where many keys map to the same index, increasing the likelihood of collisions and degrading performance.
- ☐ B) A bad hash function will always result in an $O(n)$ search time, regardless of the collision resolution strategy.
- ☐ C) Simple hash functions always generate unique indices, so collisions do not occur.
- ☐ D) A poor hash function does not affect the performance, as it only impacts the deletion operation.



Which of the following collision resolution strategies uses a secondary hash function to resolve collisions?

* 1 point

In hash tables, various techniques are used to handle cases where multiple keys hash to the same index. One technique involves using a second hash function to further refine the probe sequence.

- ☐ A) Linear probing
- ☐ B) Quadratic probing
- ☐ C) Double hashing
- ☐ D) Chaining

In linear probing, what happens when a collision occurs at a particular index in a hash table?

* 1 point

Linear probing resolves collisions by sequentially searching for the next available slot, starting from the point of collision and checking subsequent indices.

- ☐ A) The element is placed in the next available slot, checking all indices until an empty one is found.
- ☐ B) The element is placed in the next index, and if a collision occurs again, the table is resized.
- ☐ C) The collision is resolved by applying a secondary hash function.
- ☐ D) The hash table automatically expands to accommodate more elements when a collision occurs.



Which of the following is true about deleting an element from a hash table when using open addressing with linear probing? * 1 point

Deleting an element from a hash table can be tricky with open addressing because it can affect the integrity of the probe sequence.

- ☐ A) The deleted element is simply removed, and the following elements are not affected.
- ☐ B) To avoid breaking the probe chain, the deleted element is marked as deleted (using a special "deleted" marker) rather than completely removing it.
- ☐ C) The table is rehashed every time an element is deleted to maintain efficiency.
- ☐ D) Deleting an element in linear probing is equivalent to shifting all following elements one position to the left.

Which of the following is NOT a characteristic of a graph in graph theory? * 1 point

- ☐ A) It consists of vertices and edges
- ☐ B) Edges are always directed
- ☐ C) A graph can have cycles
- ☐ D) A graph can have multiple disconnected components

What is the time complexity of accessing an element in an adjacency matrix representation of a graph? * 1 point

- ☐ A) $O(1)$
- ☐ B) $O(n)$
- ☐ C) $O(n^2)$
- ☐ D) $O(\log n)$



Which of the following graph traversal algorithms explores all the vertices at the current level before moving to the next level? * 1 point

- ☐ A) Depth First Search
- ☐ B) Breadth First Search
- ☐ C) Dijkstra's Algorithm
- ☐ D) Floyd-Warshall Algorithm

Which of the following is a true statement about Depth First Search (DFS)? * 1 point

- ☐ A) DFS explores all neighbors of a vertex before moving to a new vertex
- ☐ B) DFS uses a queue to store the vertices to be explored
- ☐ C) DFS is typically implemented using a breadth-first queue
- ☐ D) DFS cannot be used to find a cycle in a graph

What is the main advantage of using an adjacency list representation over an adjacency matrix for sparse graphs? * 1 point

- ☐ A) Adjacency lists use less memory in sparse graphs
- ☐ B) Adjacency lists are faster to traverse
- ☐ C) Adjacency lists are always more efficient for finding the shortest path
- ☐ D) Adjacency lists support both directed and undirected graphs simultaneously



In which scenario would Dijkstra's algorithm fail to compute the shortest path correctly? * 1 point

- ☐ A) When there are negative edge weights
- ☐ B) When the graph is not connected
- ☐ C) When the graph is cyclic
- ☐ D) When the graph contains multiple edges between two vertices

Which of the following algorithms is used for finding the shortest path between all pairs of vertices in a graph? * 1 point

- ☐ A) Dijkstra's Algorithm
- ☐ B) Prim's Algorithm
- ☐ C) Floyd-Warshall Algorithm
- ☐ D) Bellman-Ford Algorithm

In Prim's algorithm for finding a Minimum Spanning Tree (MST), which data structure is commonly used to keep track of the vertices to be added to the MST? * 1 point

- ☐ A) Queue
- ☐ B) Stack
- ☐ C) Priority Queue
- ☐ D) Hash Map



When using Kruskal's algorithm to find the Minimum Spanning Tree (MST), * 1 point
which data structure is typically employed to detect cycles in the graph?

- ☐ A) Linked List
- ☐ B) Disjoint Set (Union-Find)
- ☐ C) Array
- ☐ D) Binary Heap

Which of the following is the key property of the Floyd-Warshall algorithm?

* 1 point

- ☐ A) It finds the shortest path from a source vertex to all other vertices
- ☐ B) It can handle graphs with negative edge weights
- ☐ C) It works only for directed graphs
- ☐ D) It requires the graph to be connected

Which of the following is NOT a fundamental technique in algorithm design?

* 1 point

- ☐ A) Divide and Conquer
- ☐ B) Greedy Algorithms
- ☐ C) Binary Search
- ☐ D) Backtracking



: In the context of asymptotic analysis, which of the following notations represents the upper bound of an algorithm's complexity?

* 1 point

- ☐ A) $O(n)$
- ☐ B) $\Omega(n)$
- ☐ C) $\Theta(n)$
- ☐ D) $O(n^2)$

Which of the following is a characteristic feature of a greedy algorithm? *

1 point

- ☐ A) It solves the problem by breaking it down into subproblems
- ☐ B) It always makes the locally optimal choice with the hope of finding a global optimum
- ☐ C) It uses recursion to solve problems
- ☐ D) It evaluates all possible solutions before making a decision

What is the primary advantage of using dynamic programming over brute-force algorithms?

* 1 point

- ☐ A) It reduces time complexity by storing intermediate results
- ☐ B) It guarantees finding the optimal solution in every case
- ☐ C) It solves problems in a top-down fashion
- ☐ D) It uses more space to store results than brute-force algorithms



Which of the following is true about the backtracking algorithm? *

1 point

- ☐ A) It is used for optimization problems where a global optimum must be found
- ☐ B) It explores all possible solutions, but it only returns the first valid one
- ☐ C) It systematically searches through the solution space by trying to build a solution incrementally
- ☐ D) It always produces the optimal solution by using a greedy approach

Which of the following best describes the time complexity of a divide and conquer algorithm in the case of a recurrence relation like $T(n) = 2T(n/2) + O(n)$? *

1 point

- ☐ A) $O(n \log n)$
- ☐ B) $O(n^2)$
- ☐ C) $O(n)$
- ☐ D) $O(\log n)$

When performing an analysis of algorithms, which of the following best describes time complexity?

* 1 point

- ☐ A) It measures the amount of space an algorithm uses
- ☐ B) It measures the number of operations an algorithm performs as a function of input size
- ☐ C) It measures how fast an algorithm solves a problem
- ☐ D) It measures how memory usage varies with different inputs



In which of the following situations is a branch-and-bound algorithm typically used?

* 1 point

- ☐ A) To find the shortest path in a graph with negative weights
- ☐ B) To solve optimization problems, especially where searching the entire solution space is impractical
- ☐ C) To solve problems that can be broken into independent subproblems
- ☐ D) To find a solution in an environment where the problem's solution space is small

Which of the following is the key advantage of stochastic algorithms over deterministic algorithms?

* 1 point

- ☐ A) Stochastic algorithms always find the optimal solution
- ☐ B) Stochastic algorithms are simpler to implement than deterministic algorithms
- ☐ C) Stochastic algorithms introduce randomness to find a good solution when deterministic algorithms are impractical
- ☐ D) Stochastic algorithms have lower time complexity than deterministic algorithms

Which of the following is the main objective of a brute-force algorithm?

* 1 point

- ☐ A) To find the optimal solution in an efficient manner
- ☐ B) To explore all possible solutions without using any heuristics
- ☐ C) To break down the problem into subproblems and solve each one independently
- ☐ D) To use a divide and conquer approach to reduce the size of the problem



What is the primary difference between time complexity and space complexity?

* 1 point

- ☐ A) Time complexity refers to the number of operations performed, while space complexity refers to the memory used
- ☐ B) Time complexity is only relevant for recursive algorithms, while space complexity applies to iterative algorithms
- ☐ C) Time complexity is about how quickly the algorithm runs, and space complexity is about how the algorithm handles input size
- ☐ D) Time complexity measures the number of inputs, and space complexity measures the output size

Which of the following is an advantage of using an array as a data structure?

* 1 point

- ☐ A) Dynamic size
- ☐ B) Random access to elements
- ☐ C) Easy insertion and deletion
- ☐ D) Memory efficiency

In the context of Big O notation, which of the following is true for an algorithm with time complexity $O(n^2)$?

* 1 point

- ☐ A) The time complexity increases linearly as the input size increases
- ☐ B) The time complexity grows quadratically as the input size increases
- ☐ C) The time complexity grows exponentially as the input size increases
- ☐ D) The time complexity remains constant for large input sizes



Which of the following operations in a stack is performed in constant time?

* 1 point

- ☐ A) Push
- ☐ B) Pop
- ☐ C) Peek
- ☐ D) All of the above

Which of the following is the correct way to represent a queue using an array?

* 1 point

- ☐ A) Using two pointers, one for the front and one for the rear
- ☐ B) Using only one pointer for both operations
- ☐ C) Using a single stack for enqueue and dequeue operations
- ☐ D) Using a circular array

In a singly linked list, what happens when a new node is inserted at the beginning?

* 1 point

- ☐ A) The new node becomes the last node
- ☐ B) The new node becomes the first node and the head pointer is updated
- ☐ C) The new node is added at the end of the list
- ☐ D) The new node is inserted before the last node



Which of the following is a disadvantage of using a doubly linked list over a singly linked list? * 1 point

- ☐ A) Increased memory usage due to the additional pointer
- ☐ B) More complex implementation
- ☐ C) Slower traversal
- ☐ D) Both A and B

What is the time complexity of accessing an element in a linked list? * 1 point

- ☐ A) $O(1)$
- ☐ B) $O(\log n)$
- ☐ C) $O(n)$
- ☐ D) $O(n^2)$

In recursion, what is the role of the base case? * 1 point

- ☐ A) To define the recursive function
- ☐ B) To prevent infinite recursion and return a result
- ☐ C) To call the function recursively
- ☐ D) To keep track of memory usage

Submit

Clear form

Never submit passwords through Google Forms.

This content is neither created nor endorsed by Google. - [Contact form owner](#) - [Terms of Service](#) - [Privacy Policy](#).



Does this form look suspicious? [Report](#)

Google Forms



